

آزمایشگاه سیستم عامل

پروژه چهارم : همگام سازی در xv6

محمد امین رشید 810102454

سید محمد جواد شریفی 810102469

محمد صادق مهدوی پور 810102519

بخش اول : تحلیل هسته و قفل های پیشرفته

سوالات تشریحی:

ج1) مسیر کنترلی (control path) ، جریان اجرای کد در CPU است که از نقطه‌ای شروع می‌شود و تا نقطه‌ای دیگر ادامه پیدا می‌کند (منظور همان ترتیبی از instruction هاست). این مسیر می‌تواند شامل تغییر مود پردازنده از حالت کرنل به یوزر یا برعکس باشد.

مسیر کنترلی سیستم کال : مسیری است که توسط یک پردازنده کاربر ایجاد میشود تا سرویس کرنل را درخواست کند. این مسیر از پردازنده کاربر نشأت می‌گیرد و پیش بینی پذیر هست و از حالت یوزر به کرنل می‌رویم.

مسیر کنترلی وقفه : ناشی از رخداد ناگهانی سخت افزاری که cpu را مستقل از وضعیت پراسس ، وادار به انجام یکسری دستورات خاص میکند و وارد یک مسیر کنترلی میکند. در این حالت منشا سخت افزار است و از حالت یوزر به کرنل می‌رود و یا اگر در کرنل بوده در آنجا میماند. (مانند I/O)

مسیر کنترلی استثنا : در این حالت ، خطایی در خود cpu رخ میدهد مثل تقسیم بر 0 و باز هم ممکنه در هر 2 حالت یوزر و کرنل این رخ بدهد. استثنا ممکن است موجب سیگنال و یا terminate شدن پردازنده فعلی شود.

مهم ترین تفاوت بین این 3 تا این است که syscall توسط کاربر است و پیش بینی پذیره برخلاف وقفه و استثنا که میتوانند پردازنده را از مسیر طبیعی خودش خارج کنند.

ج2) به هسته ای Reentrant می‌گویند که اجازه میدهد که چندین مسیر کنترلی به صورت همزمان در کد هسته اجرا شوند بدون اینکه داده های مشترک خراب شوند. این چندین مسیر میتوانند وقفه ها باشند برای مثال.

اگر یک پراسس در حال رسیدگی به وقفه باشد (در حال اجرای syscall باشد) ، و همان لحظه دیسک یک وقفه دیگر بدهد. فرض میکنیم که سیستم کال write() در حال اجرا بوده است و در حال نوشتن بر روی بافر یا هر دیتای دیگری در کرنل بوده است ، وقتی به سراغ وقفه ای که از دیسک آمده می‌رویم ، ممکن است لازم شود همان بافر دستکاری شود مثلا بخواند داده های بافر را به حافظه فیزیکی منتقل کنه و باعث میشود که دیتا ها خراب شوند.

هم چنین اگر دو تا روال وقفه نیاز به یک قفل مشترک داشته باشند ، منجر به وضعیت بن بست میشود. یک راه حل میتونه برای این وضعیت غیرفعال کردن وقفه ها در مسیر بحرانی در کرنل باشد.

ج3) هر مسیر کنترلی در یک context خاص اجرا میشود:

Process context یعنی پردازنده در حال اجرای کد یک پراسس در kernel mode باشد. در این حالت مجاز به sleep() و wait() و مسدود شدن هستیم و مجاز به context switch هستیم. در این حالت cpu متعلق به پردازنده است و چون برنامه کاربر است ، مدت زمانش میتواند طولانی باشد. پس اجرای کد سیستم کال و یا استثنا در process context رخ میدهد.

Interrupt context یعنی پردازنده در حال اجرای یک Handler وقفه می باشد. در این حالت cpu به هیچ پردازنده ای تعلق ندارد و برخلاف process context نباید block و sleep بشود . هم چنین امکان context switch نداریم چون عملا پراسسی در cpu در حال اجرا نیست که بخواهیم عوضش کنیم. مدت زمان اجرا در این حالت معمولا سریع است و handler نباید زیاد طول بکشد و وقفه ها هم نیز در این زمان کوتاه معمولا غیر فعال میشوند. بطور خلاصه اجرای کد وقفه در این بخش صورت میگیرد.

ج4) در حالتی که cpu در حال رسیدگی به وقفه هست نباید مسدود بشه یا به sleep برود زیرا سناریو های متفاوتی ممکن است رخ بدهد مثلا اگر سیستم تک هسته ای باشد منجر به بن بست میشود یا حتی در حالت چند هسته ای نیز ممکن است پراسس دیگری این وقفه را بیدار نکند و باز هم بن بست رخ بدهد. هم چنین علاوه بر مشکل deadlock ممکن است سایر وقفه ها غیرفعال شوند. هم چنین cpu در این حالت که در حال رسیدگی به وقفه به خواب برود عملا cpu درگیر میشود و کار مفیدی انجام نمیدهد(در حالت process context میتوان فرآیند را تعویض کرد ولی اینجا امکانش نیست)

یک سناریو میتوان برای هندلر تایمر فرض کرد که مسدود بشود ، و این باعث میشود که زمان سیستم جلو نرود و عملا الگوریتم scheduling کار نکند زیرا تایمر وقفه از کار افتاده است و باعث میشه که به وضعیت deadlock برسیم.

ج5) spinlock به تنهایی فقط میتواند جلوی race condition را بگیرد یعنی 2 تا پراسس همزمان وارد مسیر بحرانی نشوند ولی در interrupt context ممکنه با وجود این قفل deadlock رخ بدهد اگر وقفه غیرفعال نباشد زیرا handler به قفلی نیاز دارد که توسط پراسس دیگری گرفته شده است . هم چنین غیرفعال کردن وقفه به تنهایی کافی نیست زیرا در سیستم چند هسته ای ممکن با وجود غیرفعال بودن وقفه ، 2 تا پردازنده در اجرای روال وقفه به یک داده مشترک دسترسی پیدا کنند و باعث خرابی دیتا شوند.

Spinlock : پراسس تا آزاد شدن قفل روی یک حلقه میماند (busy wait) و بعد از اینکه قفل آزاد شد ، از این لوپ خارج میشود. از مزایا میتوان به این اشاره کرد که برای انتظار های کوتاه خیلی مناسب است زیرا دیگر نیاز به تایم اضافه برای contex switch نیستیم و سریع انجام میشود. هم چنین پیاده سازی راحتی دارد.

معایبش هم برای زمانی است که انتظار طولانی شود، و این یعنی تایم مفید cpu هدر میرود. هم چنین در صورت پیاده سازی نادرست میتواند منجر به livelock شود که شبیه بن بست است با این تفاوت بدون اینکه متوقف شوند ، سیستم پیشروی نمی کند.

Sleeplock : پراسس وقتی به قفلی میرسد (درخواست قفل) که قفل آزاد نباشد ، به خواب میرود تا زمانی که قفل آزاد بشود مثل سمافور که وقتی که ریسورس خالی نباشد، به خواب میرود و هنگام (signal) بیدار میشود. مزیت قفل برای وقتی است که انتظار برای گرفتن قفل زیاد باشد (بیش از 2 برابر زمان contex switch) و باعث مصرف بهینه cpu میشود. عیبش هم پیاده سازی پیچیده تر (نیاز به صف و...) و overhead تعویض متن است که البته به صرفه تر است.

Lock-free Programming : از قفل استفاده نمیشود و با عملیات atomic از race condition جلوگیری میکند. مزایا اش این است که خطر بن بست رو نداریم. بزرگ ترین عیبش پیاده سازی بسیار پیچیده اش است. هم چنین عیب دیگرش میتواند این باشد که در محیط های شلوغ ، ممکن است عملیات اتمیک چندین بار شکست بخورد و مصرف cpu را بالا ببرد.

(ج7) مشابه توضیحاتم در بخش 4 و 5 ، اگر وقفه غیر فعال نشود و وقفه ای رخ بدهد و بخواد قفلی را بگیرد که در دست پراسس بوده است 2 حالت دارد اگر سیستم تک هسته ای باشد بن بست رخ میدهد زیرا اولویت با وقفه است و تا زمانی که کارش تمام نشده ، پراسس نمیتونه قفل رو آزاد کنه. در سیستم چند هسته ای هم ، احتمال آزاد شدن قفل وجود دارد ولی با این حال باعث کندی عملکرد وقفه میشود در حالی که ما میخواهیم هندلر وقفه سریع و بدون اتلاف انجام شود پس با تابع cli قبل گرفتن قفل وقفه هارو غیر فعال میکنیم.

(ج8) برای مدیریت وقفه ها در حالت nested یعنی مثلا مسیری پیش بیاید که چندین بار cli فراخوانی شود ، در این حالت باید به همان تعداد sti فراخوانی شود تا وقفه فعال شود. که این مکانیزم توسط توابع pushcli و popcli در xv6 پیاده سازی شده است. همانطور که از کد معلوم است ، cpu فیلدی به نام ncli دارد که همین تعداد cli ها پشت سر هم را جمع میکند و در صورت sti قبلش در تابع popcli ، از ncli یکی کم می شود.

توجه داریم که اگر وقفه ها فعال باشند و popcli فراخوانی شود ، سیستم پنیک میکند. از فیلد intena هم برای ذخیره حالت فعلی وقفه ها وقتی که برای بار اول pushcli میکنیم، استفاده می کنیم.

```
99
100 // Pushcli/popcli are like cli/sti except that they are matched:
101 // it takes two popcli to undo two pushcli. Also, if interrupts
102 // are off, then pushcli, popcli leaves them off.
103
104 void
105 pushcli(void)
106 {
107     int eflags;
108
109     eflags = readeflags();
110     cli();
111     if(mycpu()->ncli == 0)
112         mycpu()->intena = eflags & FL_IF;
113     mycpu()->ncli += 1;
114 }
115
116 void
117 popcli(void)
118 {
119     if(readeflags() & FL_IF)
120         panic("popcli - interruptible");
121     if(--mycpu()->ncli < 0)
122         panic("popcli");
123     if(mycpu()->ncli == 0 && mycpu()->intena)
124         sti();
125 }
126
```

بخش عملی:

pid موجود در ساختار قفل نشان دهنده پراسس آیدی مالک قفل است که در تابع acquires مقدار صاحبش را میگیرد و در release چک میکند کسی که ریلیزش میکند آیا همان صاحب قفل است ؟ اگر نباشد پنیک میکند.

```
1 // Long-term locks for processes
2 struct sleeplock {
3     uint locked; // Is the lock held?
4     struct spinlock lk; // spinlock protecting this sleep lock
5
6     // For debugging:
7     char *name; // Name of lock.
8     int pid;
9 };
10
```

```

20 void initsleeplock(struct sleeplock *lk, char *name)
21 {
22     initlock(&lk->lk, "sleep lock");
23     lk->name = name;
24     lk->locked = 0;
25     lk->pid = 0;
26 }
27
28 void acquiresleep(struct sleeplock *lk)
29 {
30     acquire(&lk->lk);
31     while (lk->locked)
32     {
33         sleep(lk, &lk->lk);
34     }
35     lk->locked=1;
36     lk->pid = myproc()->pid;
37     release(&lk->lk);
38 }
39
40 void releasesleep(struct sleeplock *lk)
41 {
42     acquire(&lk->lk);
43     if (lk->pid != myproc()->pid)
44     {
45         panic("Panic in releasesleep !!: This process isn't the owner of this lock!");
46     }
47     lk->locked = 0;
48     lk->pid = 0;
49     wakeup(lk);
50     release(&lk->lk);
51 }

```

توجه داریم که برای گرفتن و آزاد کردن قفل نیاز داریم در سطح کرنل صورت بگیره پس سیستم کال های زیر را اضافه میکنیم و هم چنین تابع check and init برای این است که اولین دفعه ای که یکی از این 2 تا سیستم کال فراخوانی میشود ، قفل ایجاد شود. این تابع در sleeplock.c میگذاریم و سیستم کال هارا در sysproc.c.

```

int sys_test_lock_acquire(void)
{
    check_and_init_test_lock();
    acquiresleep(&test_lock);
    return 0;
}

int sys_test_lock_release(void)
{
    check_and_init_test_lock();
    releasesleep(&test_lock);
    return 0;
}

```

```

15
16 void check_and_init_rw()
17 {
18     if (rw_initied==0)
19     {
20         rwlock_init(&global_rw, "test_rw");
21         rw_initied = 1;
22     }
23 }

```

```

2
3 struct sleeplock test_lock;
4 int test_lock_initied = 0;
5

```

برای تست ، پدر فرزندی را ایجاد میکند ، فرزند 10 تیک صبر میکند تا والد قفل را بگیرد بعدش برای گرفتن قفل تلاش میکند که باعث پنیک سیستم میشود همانطور که نوشته بودیم.

```

SeaBIOS (version 1.16.3-debian-1.16.3-2)
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFCB050+1EF0B050 CA00
Booting from Hard Disk...
cpu0: starting 0
fsb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap sta
t 58
Sadegh Mahdavi
javad sharifi
Mohammad Amin Rashid
init: starting sh
$ test_sleep
Child is trying to release lock of parent !!
lapicid 0: panic: Panic in releasesleep !!! This process isn't the owner of this
lock!
80105f6a 80107708 801068e9 801079ed 801077a1 0 0 0 0 0_

```

LAB4 > xv6 > `test_sleep.c` > `main()`

```

1
2  int main() {
3      printf (1, "\n");
4      int pid=fork();
5      if (pid==0) {
6          sleep(10);
7          printf(1,"Child is trying to release lock of parent !!\n");
8          test_lock_release();
9          exit();
10     }
11     test_lock_acquire();
12     wait();
13     // test_lock_release();
14     exit();
15 }

```

این در حالست که اگر بخش کامنت شده را به جای عملیات فرزند اعمال کنیم ، قفل به درستی آزاد میشود زیرا خود مالک قفلی را که گرفته آزاد میکند.

بخش امتیازی:

```
0
1 struct rwlock {
2     struct spinlock lk;
3     int read_count;
4     int writing;
5     char *name;
6 };
7
```

در این بخش 5 تا تابع پیشنهادی را مشابه sleeplock می نویسیم:

Lk همان قفل چرخشی درونی برای محافظت از خود مقادیر این ساختار در این چند تابع است.

Read_count تعداد خوانندگان است. writing نشان میدهد که آیا قفل دست نویسنده است؟ این همان قفل اصلی باینری مان است.

وقتی خواننده قفلی را می گیرد، صرفا باید writing 0 باشد. وقتی هم که قفل رو رها میکنه باید چک کنه اگه تعداد خواننده ها 0 شده باشد، نویسنده منتظر قفل رو بگیره.

وقتی نویسنده میخواهد قفل رو بگیرد باید تا زمانی که نویسنده دیگری یا خواننده دیگری قفل رو دارد به خواب برود و موقع رها کردن قفل writing رو 0 میکنه و wakeup رو صدا میزنه که اگر کس دیگری در sleep بوده قفل رو بگیرد.

```

5 void rwlock_init(struct rwlock *rw, char *name)
6 {
7     initlock(&rw->lk, "rwlock");
8     rw->read_count = 0;
9     rw->writing = 0;
10    rw->name = name;
11 }
12
13 void rwlock_acquire_read(struct rwlock *rw)
14 {
15     acquire(&rw->lk);
16     while (rw->writing)
17     {
18         sleep(rw, &rw->lk);
19     }
20     rw->read_count++;
21     release(&rw->lk);
22 }
23
24 void rwlock_release_read(struct rwlock *rw)
25 {
26     acquire(&rw->lk);
27     rw->read_count--;
28     int readers=rw->read_count;
29     if (readers==0)
30     {
31         wakeup(rw);
32     }
33     release(&rw->lk);
34 }
35
36 void rwlock_acquire_write(struct rwlock *rw)
37 {
38     acquire(&rw->lk);
39     while ((rw->writing==1) || (rw->read_count>0))
40     {
41         sleep(rw, &rw->lk);
42     }
43     rw->writing=1;
44     release(&rw->lk);
45 }
46
47 void rwlock_release_write(struct rwlock *rw)
48 {
49     acquire(&rw->lk);
50     rw->writing=0;
51     wakeup(rw);
52     release(&rw->lk);
53 }

```

مشابه بخش قبل سیستم کال و توابع مورد نیاز رو پیاده سازی میکنیم:

```

int sys_rw_acquire_read(void)
{
    check_and_init_rw();
    rwlock_acquire_read(&global_rw);
    return 0;
}

int sys_rw_release_read(void)
{
    check_and_init_rw();
    rwlock_release_read(&global_rw);
    return 0;
}

int sys_rw_acquire_write(void)
{
    check_and_init_rw();
    rwlock_acquire_write(&global_rw);
    return 0;
}

int sys_rw_release_write(void)
{
    check_and_init_rw();
    rwlock_release_write(&global_rw);
    return 0;
}

```

```

void check_and_init_rw()
{
    if (rw_initied==0)
    {
        rwlock_init(&global_rw, "test_rw");
        rw_initied = 1;
    }
}

```

سپس قفل رو تست میکنیم:

```

LAB4 > xv6 > c test_rw.c > main()
1  #include "types.h"
2  #include "stat.h"
3  #include "user.h"
4
5  int main()
6  {
7      printf(1, "\n");
8
9      if (fork()==0)
10     {
11         printf(1, "witer 1 try to get lock\n");
12         rw_acquire_write();
13         printf(1, "writer 1 entered!\n");
14         sleep(40);
15         printf(1, "writer 1 leaved\n");
16         rw_release_write();
17         exit();
18     }
19     if (fork()==0)
20     {
21         sleep(5);
22         printf(1, "witer 2 try to get lock\n");
23         rw_acquire_write();
24         printf(1, "writer 2 entered!\n");
25         sleep(40);
26         printf(1, "writer 2 leaved\n");
27         rw_release_write();
28         exit();
29     }
30     for (int i =1; i<4; i++)
31     {
32         if (fork()==0)
33         {
34             sleep(80+i*3);
35             printf(1, "reader %d try to get lock\n",i);
36             rw_acquire_read();
37             printf(1, "reader %d entered!\n",i);
38             sleep(40);
39             printf(1, "reader %d: Leaved\n",i);
40             rw_release_read();
41             exit();
42         }
43     }
44     if (fork()==0)
45     {
46         sleep(100);
47         printf(1, "witer 3 try to get lock\n");
48         rw_acquire_write();
49         printf(1, "writer 3 entered!\n");
50         sleep(40);
51         printf(1, "writer 3 leaved\n");
52         rw_release_write();
53         exit();
54     }
55
56     for (int i=0; i<6; i++)
57     {
58         wait();
59     }
60
61     printf(1, "FINISH\n");
62     exit();
63 }
64

```

در این برنامه ابتدا نویسنده اول قفل رو میگیره و سپس نویسنده دوم سعی میکنه قفل رو بگیرد ولی موفق نمیشود ، زمانی که کار نویسنده اول تمام میشود نویسنده دوم قفل رو بالاخره میتونه بگیره و کارش را تمام میکند.

سپس 3 تا خواننده ایجاد میکنیم، این ها به ترتیب قفل رو میگیرند،(چند خواننده هم زمان ایراد نداره) در این حین که این 3 تا خواننده قفل رو دارند، نویسنده سوم سعی در گرفتن قفل میکند ولی موفق نمیشه و زمانی موفق میشه که هر 3 تا خواننده کارشان را تمام کنند.

```
Machine View
javad sharifi
Mohammad Amin Rashid
init: starting sh
$ test_rw
witer 1 try to get lock
writer 1 entered!
witer 2 try to get lock
writer 1 leaved
writer 2 entered!
writer 2 leaved
reader 1 try to get lock
reader 1 entered!
reader 2 try to get lock
reader 2 entered!
reader 3 try to get lock
reader 3 entered!
witer 3 try to get lock
reader 1 Leaved
reader 2 Leaved
reader 3 Leaved
writer 3 entered!
writer 3 leaved
FINISH
$ _
```

بخش دوم: مقیاس پذیری، داده های Per-CPU و پروفایلینگ قفل ها در xv6 :

سوالات تئوری:

س ۱) دلیل اصلی این کار جلوگیری از بن بست است (deadlock). اگر وقفه ها فعال باشند و پردازنده قفلی را در اختیار داشته باشد، وقوع وقفه باعث اجرای Interrupt Handler میشود. اگر این تلاش کند همان قفل را بگیرد وارد یک حلقه انتظار (spin) میشود و چون Interrupt Handler اولویت بالاتری دارد و پردازش کدی که قفل را در اختیار دارد را متوقف کرده است قفل هرگز آزاد نمیشود و سیستم روی همان پردازنده قفل میکند.

س ۲) این وضعیت باعث ایجاد یک بن بست روی یک پردازنده واحد میشود (Single CPU deadlock) مثال و توضیح: پروسه P یک قفل را در اختیار دارد. در حین اجرا، وقفه رخ میدهد و پردازنده به سراغ اجرای Interrupt Handler میرود و اجرای پروسه P را متوقف میکند (چون هندلر وقفه اولویت بالاتر دارد). هندلر تلاش میکند قفل را بگیرد ولی چون قفل آزاد نیست می چرخد و منتظر آزاد شدن آن می ماند (spinning) اما چون هندلر تمام نشده است کنترل هرگز به پروسه P بر نمیگردد تا قفل را آزاد کند و لذا به بن بست میرسد.

س ۳) در این حالت پدیده ای به نام پینگ پونگ کش رخ میدهد. پروتکل MESI برای حفظ یکپارچگی، زمانی که یک هسته متغیر گلوبال را تغییر میدهد نسخه های آن متغیر را در cache سایر هسته ها invalid میکند. این باعث میشود بقیه هسته ها برای دسترسی به داده درست مجبور شوند داده را دوباره از حافظه اصلی یا کش هسته دیگر بگیرند و این باعث ترافیک سنگین روی bus و در نتیجه کندی شدید سیستم میشود.

س ۴) وقتی از داده های Per-CPU استفاده میکنیم، هر هسته نسخه اختصاصی خود را از شمارنده دارد که در خطوط کش جداگانه ذخیره میشود. بنابراین وقتی یک هسته متغیر خود را تغییر میدهد، نیازی به invalid کردن کش سایر هسته ها ندارد (چون دیگران روی آن آدرس خاص نمی نویسند). این کار ترافیک bus را حذف کرده و کارایی را بشدت افزایش میدهد. تنها در زمان خواندن نهایی نیاز به جمع بندی مقادیر است.

س ۵) قفل چرخشی یا همان spinlock از روش انتظار مشغول استفاده میکند (busy waiting). پردازنده در یک حلقه مداوم وضعیت قفل را چک میکند و تا زمان آزاد شدن آن سیکل های پردازنده را مصرف میکند. (مناسب برای انتظار های کوتاه)

قفل خواب یا همان sleeplock پردازنده را واگذار میکند. اگر قفل اشغال باشد پروسه به خواب میرود و سیستم عامل پردازنده را به پروسه دیگری اختصاص میدهد (Context Switch). پس از آزاد شدن قفل پروسه بیدار میشود (مناسب برای انتظار های طولانی).

توضیح قسمت عملی:

ابتدا به ساختار spinlock دو آرایه توضیح داده شده در پروژه را می افزاییم:

```
// Mutual exclusion lock.
struct spinlock {
    uint locked;           // Is the lock held?

    // For debugging:
    char *name;            // Name of lock.
    struct cpu *cpu;       // The cpu holding the lock.
    uint pcs[10];          // The call stack (an array of program counters)
                           // that locked the lock.

    uint acq_count[NCPU];
    uint total_spins[NCPU];
};
```

حالا طبق صورت پروژه باید ابتدا همه مقادیر این دو آرایه صفر شوند (در تابعی که قفل ها مقدار دهی اولیه می شوند):

```
void
initlock(struct spinlock *lk, char *name)
{
    lk->name = name;
    lk->locked = 0;
    lk->cpu = 0;

    for(int i = 0; i < NCPU; i++){
        lk->acq_count[i] = 0;
        lk->total_spins[i] = 0;
    }
}
```

حالا در تابعی که مسئول گرفتن قفل است متغیر spins را برای شمارش تعداد دوره های انتظار تعریف می کنیم و در حلقه هر بار مقدار آن را یکی افزایش می دهیم. در آخر پس از گرفتن قفل شناسه پردازنده را می گیریم و متغیر acq_count آن را یکی افزایش داده و متغیر total_spins آن را به اندازه متغیر محلی spins زیاد می کنیم.

```
void
acquire(struct spinlock *lk)
{
    pushcli(); // disable interrupts to avoid deadlock.
    if(holding(lk))
        panic("acquire");

    uint spins = 0;
    // The xchg is atomic.
    while(xchg(&lk->locked, 1) != 0)
        spins++;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen after the lock is acquired.
    __sync_synchronize();

    // Record info about lock acquisition for debugging.
    int id = cpuid();
    lk->acq_count[id]++;
    lk->total_spins[id] += spins;

    lk->cpu = mycpu();
    getcallerpcs(&lk, lk->pcs);
}
```

حالا طبق صورت پروژه سیستم کال مربوطه را تعریف می کنیم:

```

extern struct spinlock tickslock;

int
sys_getlockstat(void)
{
    char *score_ptr;
    uint scores[NCPU];
    struct spinlock *lk = &tickslock;

    if(argptr(0, &score_ptr, sizeof(scores)) < 0) {
        cprintf("DEBUG: argptr FAILED!\n");
        return -1;
    }

    // cprintf("\n--- KERNEL DEBUG STATS ---\n");
    for(int i = 0; i < NCPU; i++){
        // cprintf("CPU %d: Acq = %d, Spins = %d\n", i, (int)lk->acq_count[i], (int)lk->total_spins[i]);
        if(lk->total_spins[i] > 0) {
            scores[i] = lk->total_spins[i] / lk->acq_count[i];
        } else {
            scores[i] = lk->acq_count[i];
        }
    }

    if(copyout(myproc()->pgdir, (uint)score_ptr, (void*)&scores, sizeof(scores)) < 0)
        return -1;

    return 0;
}

```

در فایل های user.h, syscall.c, syscall.h و usys.S به ترتیب:

```
#define SYS_getlockstat 30
```

```
extern int sys_getlockstat(void);
```

```
[SYS_getlockstat] sys_getlockstat,
```

```
int getlockstat(uint *score);
```

```
SYSCALL(getlockstat)
```

در آخر برنامه سطح کاربر را با توجه به توضیحات صورت پروژه پیاده سازی میکنیم:

```

int
main(int argc, char *argv[])
{
    uint scores[NCPU];

    printf(1, "\nStarting Lock Contention Test...\n");

    if(getlockstat(scores) < 0) {
        printf(1, "ERROR: System call failed!\n");
        exit();
    }

    print_scores("Initial Stats", scores);

    int n_children = 4;
    int i;
    for(i = 0; i < n_children; i++) {
        int pid = fork();
        if(pid == 0) {
            int j;
            for(j = 0; j < 1000; j++)
                sleep(1);
            exit();
        }
    }

    for(i = 0; i < n_children; i++)
        wait();

    getlockstat(scores);
    print_scores("Final Stats", scores);

    exit();
}

```

```

void
print_scores(char *label, uint *scores) {
    printf(1, "%s (Spins per Acquisition):\n", label);
    int i;
    for(i = 0; i < NCPU; i++) {
        int val = (int)scores[i];
        printf(1, "CPU %d Cost: %d spins/lock\n", i, val);
    }
}

```

خروجی به صورت زیر است:

```

init: starting sh
$ locktest
Starting Lock Contention Test...
Initial Stats (Spins per Acquisition):
CPU 0 Cost: 301 spins/lock
CPU 1 Cost: 0 spins/lock
CPU 2 Cost: 0 spins/lock
CPU 3 Cost: 0 spins/lock
CPU 4 Cost: 0 spins/lock
CPU 5 Cost: 0 spins/lock
CPU 6 Cost: 0 spins/lock
CPU 7 Cost: 0 spins/lock
Final Stats (Spins per Acquisition):
CPU 0 Cost: 438 spins/lock
CPU 1 Cost: 427 spins/lock
CPU 2 Cost: 343 spins/lock
CPU 3 Cost: 336 spins/lock
CPU 4 Cost: 0 spins/lock
CPU 5 Cost: 0 spins/lock
CPU 6 Cost: 0 spins/lock
CPU 7 Cost: 0 spins/lock

```

این خروجی نشان دهنده افزایش چشمگیر هزینه است. در ابتدا هزینه بسیار ناچیز است اما در حالت نهایی هزینه بسیار زیاد است (۴۰۰ حدوداً) که نشان میدهد هر پردازنده برای یک بار sleep کردن که نیاز به قفل tickslock دارد به طور متوسط حدود ۴۰۰ بار spin کرده است تا نوبتش بشود که به معنای اتلاف وقت پردازنده است.

از جایی که هزینه هر ۴ پردازنده تقریباً به طور یکسان بالا رفته است نشان میدهد که زمان بند سیستم عامل هوشمند بوده است و به محض (`fork()` کردن، فرزندان را به پردازنده های بیکار منتقل کرده است (در اینجا خروجی مربوط به اجرا روی ۴ پردازنده است) تا تعادل برقرار شود.

این مکانیزم ساده در مهندسی سیستم های بزرگ کاربرد دارد برای مثال:

در جایی مثل یک وب سرور یا دیتابیس واقعی هزاران قفل وجود دارد و برای حل کندی سیستم توسعه دهنده نمیداند باید کدام را بهبود بدهد. با این روش سیستم عامل کند بودن یک قفل را لو میدهد و توسعه دهنده برای بهبود سیستم مشکل را میفهمد.

بخش سوم: قفل های اولویت دار و مسئله گرسنگی

سوالات تئوری:

ج ۱) بله. در این طراحی امکان گرسنگی پردازنده های با اولویت پایین وجود دارد. وقتی سیاست سیستم این است که همواره قفل را به پردازنده ای با اولویت بالاتر بدهد، اگر جریانی از پردازنده های با اولویت بالا به صورت مداوم وارد سیستم شوند، هیچگاه نوبت به پردازنده با اولویت کمتر نمیرسد و دچار گرسنگی می شود.

مثلاً فرض کنیم که یک پردازنده با اولویت پایین (`low`) درخواست قفل می کند و چون قفل در دست پردازنده دیگری است به صف انتظار می رود (می خوابد). در همین حین و پیش از آنکه قفل آزاد شود، پردازنده دیگری با اولویت بالاتر (`high`) می رسد و این پردازنده هم به صف انتظار می رود. حال پس از آزاد شدن قفل، به دلیل سیاستی که اتخاذ کرده بودیم، نوبت به پردازنده با اولویت بالا می رسد و پردازنده با اولویت پایین کماکان منتظر می ماند. این اتفاق ممکن است بارها و بارها تکرار شود و در نتیجه هیچ تضمینی وجود ندارد که نوبت به پردازنده با اولویت پایین برسد و بنابراین دچار گرسنگی می شود.

ج ۲) قفل بلیطی (`ticket lock`) یک مکانیزم همگام سازی است که برای برقراری عدالت و رعایت ترتیب ورود طراحی شده است. این مکانیزم مانند مکانیزم نوبت دهی در نانوای ها، بانک ها و ... است و در واقع از FIFO استفاده می کند.

در این مکانیزم دو متغیر پایه داریم:

۱- شمارنده بلیط: شماره‌ای که به نفر بعدی که از راه می‌رسد داده می‌شود.

۲- شمارنده نوبت: شماره‌ای که در حال حاضر حق دسترسی به بخش بحرانی را دارد.

مراحل فرایند به این صورت است که وقتی یک پردازنده می‌خواهد وارد بخش بحرانی شود، ابتدا یک بلیط می‌گیرد. یعنی مقدار شمارنده بلیط را برای خود برمی‌دارد و مقدار آن را $1+$ می‌کند. سپس در یک حلقه منتظر می‌ماند تا زمانی که شمارنده نوبت با شماره بلیطی که در دست دارد برابر شود. در این صورت وارد بخش بحرانی می‌شود.

سپس بعد از آنکه پردازنده کارش تمام شد، شمارنده نوبت را یکی افزایش می‌دهد تا نوبت به پردازنده بعدی برسد.

ج ۳) قفل بلیطی و قفل اولویت‌دار را از ۳ جنبه مورد مقایسه قرار می‌دهیم:

۱- انصاف: قفل بلیطی عدالت و انصاف را بیشتر رعایت می‌کند. علت این است که این قفل بر اساس سیاست FIFO عمل می‌کند و قفل را در اختیار پردازنده‌ای می‌گذارد که زودتر درخواست کرده است. بنابراین وقتی یک پردازنده درخواست در اختیار گرفتن قفل را می‌کند، در صف قرار می‌گیرد و قطعاً نوبت به آن می‌رسد. اما قفل اولویت‌دار اینگونه نیست. در قفل اولویت‌دار ممکن است یک پردازنده با اولویت پایین درخواست مالکیت قفل را به سیستم بدهد اما پردازنده‌های با اولویت بالاتر هیچگاه اجازه مالکیت آن پردازنده به قفل را ندهند. در واقع در این شرایط عدالت میان پردازنده‌ها رعایت نمی‌شود.

۲- پیچیدگی: پیاده‌سازی قفل بلیطی سربار محاسباتی بسیار کمتری دارد. چرا که تنها با دو شمارنده و یک عملیات اتمی جمع قابل پیاده‌سازی است. در این مکانیزم، سربار عملیاتی از $O(1)$ انجام می‌دهیم. اما قفل اولویت‌دار نیاز به نگهداری یک صف از پردازنده‌ها دارد که هر بار که قفل آزاد می‌شود، باید کل صف را پیمایش کنیم تا پردازنده با اولویت ماکزیمم را پیدا کنیم. بنابراین در هر باری که می‌خواهیم قفل را تخصیص دهیم، باید عملیاتی از $O(n)$ انجام دهیم (n تعداد پردازنده‌های موجود در صف است). بنابراین قفل اولویت‌دار سربار محاسباتی بیشتری دارد.

۳- احتمال گرسنگی: قفل بلیطی در برابر گرسنگی ایمن‌تر است. چرا که پردازنده‌ها به ترتیب ورود، نوبت می‌گیرند و در نتیجه تضمین می‌شود که هر پردازنده، بالاخره موفق به دریافت نوبت می‌شود. اما در قفل اولویت‌دار، همانطور که در سوال ۱ توضیح داده شد، ممکن است که پردازنده‌های با اولویت پایین، هرگز قفل را دریافت نکنند و دچار گرسنگی شوند.

سوالات عملی:

ج ۱) مطابق صورت پروژه در این قسمت دو struct در فایل plock.h تعریف می‌کنیم:

```
struct plock_node {
    struct proc *process;
    int priority;
    struct plock_node *next;
};

struct plock {
    struct spinlock self_lock;
    int locked;
    struct plock_node *head;
    struct proc *owner;
};
```

plock_node به ازای هر پردازش‌ای که در صف قفل قرار می‌گیرد ساخته می‌شود و plock نیز همان قفل گلوبال اصلی است. (علاوه بر فیلدهای ذکر شده در صورت پروژه، یک فیلد به نام owner به ساختار plock اضافه شده است که پردازش مالک قفل را نگه می‌دارد. علت اضافه کردن این فیلد برای این بود که اگر به هر دلیلی چند پردازش با هم از sleep خارج شدند و خواستند قفل را بگیرند، قفل را فقط به پردازش‌ای بدهیم که واقعا مالک قفل است).

ج ۲) در این قسمت ۳ تابع را در فایل plock.c پیاده‌سازی می‌کنیم:

```
struct plock global_plock;

void plock_init(struct plock *pl)
{
    initlock(&pl->self_lock, "self_lock");
    pl->locked = 0;
    pl->owner = 0;
    pl->head = 0;
}
```

این تابع قفل اولویت‌دار گلوبال را مقداردهی اولیه می‌کند.

این تابع باید در main فراخوانی شود: plock_init(&global_plock)

```

void plock_acquire(struct plock *pl, int priority)
{
    struct proc *requesting_proc = myproc();

    acquire(&pl->self_lock);

    if(pl->locked == 0){
        pl->locked = 1;
        pl->owner = requesting_proc;
        release(&pl->self_lock);
        return;
    }

    struct plock_node *new_node = (struct plock_node*)kalloc();
    if(new_node == 0)
        panic("Error in memory allocation for plock acquire!");

    new_node->process = requesting_proc;
    new_node->priority = priority;
    new_node->next = pl->head;
    pl->head = new_node;

    while(pl->owner != requesting_proc){
        sleep(requesting_proc, &pl->self_lock);
    }

    release(&pl->self_lock);
}

```

این تابع زمانی که یک پردازش بخواند وارد بخش بحرانی شود صدا می‌شود. در ابتدا چک می‌کند که اگر قفل آزاد بود، آن را به پردازش متقاضی می‌دهد. اما اگر قفل آزاد نبود، باید پردازش را به ابتدای صف قفل اضافه کند؛ بنابراین یک `plock_node` می‌سازد و فیلدهای آن را مطابق شکل مقداردهی می‌کند. سپس تا زمانی که پردازش فعلی مالک قفل نشود، پردازش را می‌خواند.

```

void plock_release(struct plock *pl)
{
    acquire(&pl->self_lock);

    if (pl->owner != myproc())
        panic("A process other than the owner has released the lock!");

    if(pl->head == 0){
        pl->locked = 0;
        pl->owner = 0;
        release(&pl->self_lock);
        return;
    }

    struct plock_node *best = pl->head;
    struct plock_node *current = pl->head;

    while(current != 0){
        if(current->priority > best->priority){
            best = current;
        }
        current = current->next;
    }

    if(best == pl->head){
        pl->head = best->next;
    }
    else {
        struct plock_node *prev_best = pl->head;
        while(prev_best->next != best){
            prev_best = prev_best->next;
        }
        prev_best->next = best->next;
    }

    pl->locked = 1;
    pl->owner = best->process;

    wakeup(best->process);
    kfree((char*)best);

    release(&pl->self_lock);
}

```

این تابع زمانی که یک پردازنده بخواهد از بخش بحرانی خارج شود صدا زده می‌شود. در ابتدا چک می‌کند که کسی که قفل را آزاد می‌کند همان مالک قفل باشد (در غیر این صورت سیستم را abort می‌کند).

سپس اگر صف خالی باشد (یعنی $pl \rightarrow head = 0$)، قفل را آزاد کرده و return می‌کند و در غیر اینصورت در یک حلقه while، بالاترین اولویت را در بین تمام نودها پیدا می‌کند.

سپس باید این نود را حذف کند. برای این کار نود قبلی نود best را در یک حلقه while پیدا می‌کند و پوینتر next این نود را به نود بعدی best اشاره می‌دهد.

در انتها، مالک جدید قفل را تعیین کرده و پردازنده را wakeup می‌کند. این باعث می‌شود که پردازنده از حالت sleep خارج شود و اگر شرط حلقه while در تابع acquire نقض شود (یعنی این پردازنده همان مالک جدید قفل باشد)، پردازنده موفق می‌شود که از تابع acquire خارج شود و وارد بخش بحرانی شود.

ج ۳) در این قسمت دو فراخوانی سیستمی گفته شده را پیاده‌سازی می‌کنیم:

```
int sys_plock_acquire(void)
{
    int priority;
    if(argint(0, &priority) < 0)
        return -1;

    plock_acquire(&global_plock, priority);
    return 0;
}

int sys_plock_release(void)
{
    plock_release(&global_plock);
    return 0;
}
```

بدنه اصلی این سیستم‌کال‌ها را در فایل sysproc.c قرار می‌دهیم و تغییرات لازم را در فایل‌های دیگر نیز اعمال می‌کنیم.

ج ۴) در این بخش یک برنامه سطح کاربر می‌نویسیم (در فایل plock_test.c) که صحت عملکرد اولویت‌دهی را بسنجیم:

```

int main() {
    int priorities[] = {10, 20, 30, 40, 50};
    int proc_count = 5;
    int pid;

    printf(1, "\n");

    for (int i = 0; i < proc_count; i++) {
        pid = fork();
        if (pid < 0)
            exit();

        if (pid == 0) {
            int priority = priorities[i];
            if (i == 0) {
                sleep(5);
            }
            else {
                sleep(i * 20);
            }
            printf(1, "REQUEST from PID %d with priority %d\n", getpid(), priority);

            plock_acquire(priority);
            printf(1, "ACQUIRED by PID %d with priority %d\n", getpid(), priority);

            if (i == 0) {
                sleep(160);
            }
            else {
                sleep(10);
            }

            printf(1, "RELEASING by PID %d with priority %d\n", getpid(), priority);
            plock_release();

            exit();
        }
    }

    for (int i = 0; i < proc_count; i++) {
        wait();
    }
    exit();
}

```

در این برنامه، پردازش اول با اولویت ۱۰، زمان sleep کوتاهتری از بقیه پردازشها دارد. در نتیجه زودتر از بقیه بیدار شده و قفل را می‌گیرد. سپس در بخش بحرانی، به یک sleep طولانی می‌رود تا بقیه پردازشها هم در صف قرار بگیرند.

زمان sleep بقیه پردازشها طوری تنظیم شده است که هر چه اولویت کمتری داشته باشند زودتر درخواست قفل را بدهند تا عملکرد اولویت‌دهی بهتر مشخص شود.

سپس پس از آنکه بقیه پردازشها در صف قفل قرار گرفتند، پردازش اول قفل را آزاد می‌کند و بقیه پردازشها به ترتیب اولویت قفل را می‌گیرند.

خروجی اجرای برنامه به صورت زیر است:

```
$ plock_test
REQUEST from PID 4 with priority 10
ACQUIRED by PID 4 with priority 10
REQUEST from PID 5 with priority 20
REQUEST from PID 6 with priority 30
REQUEST from PID 7 with priority 40
REQUEST from PID 8 with priority 50
RELEASING by PID 4 with priority 10
ACQUIRED by PID 8 with priority 50
RELEASING by PID 8 with priority 50
ACQUIRED by PID 7 with priority 40
RELEASING by PID 7 with priority 40
ACQUIRED by PID 6 with priority 30
RELEASING by PID 6 with priority 30
ACQUIRED by PID 5 with priority 20
RELEASING by PID 5 with priority 20
$ _
```

همانطور که مشاهده می‌شود، سناریو گفته شده در صورت پروژه اجرا شده است و برنامه، قفل را به ترتیب اولویت پردازش‌ها به آنها اختصاص داده است و نه زمان درخواست آنها. چرا که مثلا پردازش ۵ با اولویت ۲۰، زودتر از پردازش ۶ با اولویت ۳۰ درخواست داده است اما قفل ابتدا به پردازش ۶ داده شده است و پس از آزادسازی به پردازش ۵ اختصاص یافته است. پس عملکرد درستی دارد.